

A Balanced Partition Framework for Concentration Synthesis on PMD Chips with Reduced Steps and Wash Overhead



Submitted by

Akansha Pandey (Roll No. 2022UCS1607)

Pallavi Sahni (Roll No. 2022UCS1616)

Diksha Sharma (Roll No. 2022UCS1590)

Under the Supervision of

Dr. Ankur Gupta

Department of Computer Science and Engineering
Netaji Subhas University of Technology, Delhi

September 2025

Acknowledgement

We would like to express our heartfelt gratitude to our supervisor, **Dr. Ankur Gupta**, for his constant guidance, encouragement, and valuable suggestions throughout the course of this project.

We also thank the Department of Computer Science and Engineering, NSUT Delhi, for providing us with the necessary resources and support.



(Dr. Ankur Gupta)

Supervisor

Department of CSE, NSUT Delhi

Contents

1	Introduction	4
1.1	Types of Biochips	4
1.2	Programmable Microfluidic Devices (PMDs)	5
2	Motivation	6
3	Literature Survey	7
3.0.1	DPMD: Design Automation for Dilution on PMD-Based Biochips	7
3.0.2	Bit-Scanning (BS) Method	7
3.0.3	Dilution Graph and De Bruijn Graph	9
3.0.4	DATS, DASDA, and DARDA: Dilution Algorithms for PMDs	9
3.0.5	Transport-Free Placement of Mixers for PMDs	11
4	Problem Statement	13
4.1	Key Performance Bottlenecks	13
4.1.1	Intermediate Usage	13
4.1.2	Wash Cycles	13
4.1.3	Mixing Steps	14
4.2	Formal Problem Definition	14
4.3	Research Significance	15
5	Objective and Methodology	16
5.1	Objectives	16
5.2	Methodology	16
6	Simulation Platform and Requirements	19
6.1	Simulation Platform	19
6.2	Implementation Platform	19
6.3	Hardware and Software Specifications	19
7	Implementation	21
7.1	System Overview	21
7.2	Environment and Dependencies	21

7.3	Core Components	21
7.4	Data Structures	21
7.5	Fixed-Array Decomposition Tree	22
7.6	Bottom-Up Merge Framework	22
7.7	Algorithm	23
7.8	Source Code Implementation	23
7.9	Experimental Output Datasets	26
8	Results and Observations	27
8.1	Mixing Complexity Trends	27
8.2	Wash Operation Efficiency	28
8.3	Sample vs Buffer Utilization	29
8.4	Average Statistical Summary Across Odd Fractions	29
8.5	Comparative Analysis with Existing Literature	30
8.5.1	Comparison Summary	31
8.5.2	Key Observations	31
8.6	Summary	32
8.6.1	Outcome	32
9	Conclusion and Future Work	33
	References	35

1. Introduction

In recent years, the field of microfluidics has witnessed remarkable growth, giving rise to devices known as *biochips* or *lab-on-a-chip* systems. These miniaturized platforms integrate multiple laboratory functions onto a single chip, making it possible to perform complex protocols such as mixing, dilution, and chemical reactions with just a few microliters of sample. By shrinking an entire laboratory to the scale of a small chip, biochips open up opportunities for rapid diagnostics, point-of-care testing, drug discovery, and environmental monitoring.

Despite their enormous promise, biochips face significant design and operational challenges. Sample preparation steps, such as mixing precise ratios, achieving target concentrations, or avoiding cross-contamination-remain difficult to automate reliably. Conventional approaches often require multiple wash cycles, excessive reagent use, and intricate routing, all of which add to chip inefficiency, increase wear on valves, and limit the scalability of current systems. Addressing these challenges is essential for translating biochip technology from controlled laboratory settings to robust, real-world applications.

1.1 Types of Biochips

Biochips can be broadly categorized based on how they handle and manipulate fluids:

- **Continuous-Flow Microfluidic Chips (CMF):** Operate using microchannels with pumps and valves to regulate fluid flow, suitable for fixed and sequential protocols.
- **Digital Microfluidic Biochips (DMF):** Manipulate discrete droplets on an electrode array using electrical signals, enabling flexible droplet-based assays.
- **MEDA-DMF Chips:** An advanced form of DMF that employs a matrix of small electrodes for fine-grained droplet control and reconfigurable operations.
- **Programmable Microfluidic Devices (PMD):** Utilize a grid of programmable valves and mixers to dynamically reconfigure fluidic pathways, offering unmatched automation and adaptability.

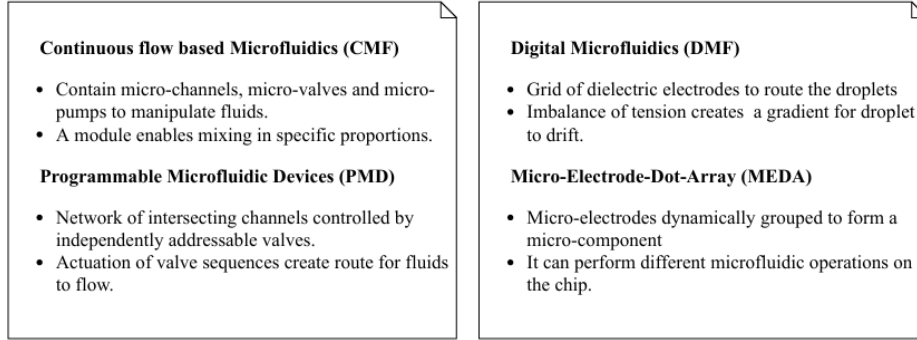


Figure 1.1: Two broad classes of biochips on basis of fluid driving force

1.2 Programmable Microfluidic Devices (PMDs)

Among these classes, PMDs stand out due to their flexibility and programmability. Each cell in a PMD grid is bordered by microvalves that can be individually controlled. This allows regions of the grid to function as channels, mixers, or storage reservoirs, depending on the experimental needs. Unlike rigid CMF or binary-mixing DMF platforms, PMDs enable arbitrary fluid routing and mixing, supporting:

- Complex dilution trees for precise concentration synthesis,
- Simultaneous execution of multiple reactions, and
- Reconfiguration for different bioassays within the same hardware.

Despite these advantages, PMDs also face limitations. Each fluid load typically requires a wash cycle to prevent contamination, and inefficient mixing sequences increase reagent waste and operational overhead. Thus, designing algorithms to optimize PMD operations is critical.

2. Motivation

The motivation for this work arises from key limitations in current PMD-based workflows:

- **Critical role of dilution and mixing:** PMDs enable precise reagent preparation by dynamically configuring grids of microvalves for fluid loading, mixing, and routing.
- **Wash cycle overhead:** Each fluid-loading step risks unintentionally filling adjacent cells. To prevent contamination, wash cycles are required after most operations, often multiple times for reused regions.
- **Impact on performance:** Excessive washing increases assay time, reagent use, and valve wear, reducing chip reliability and lifespan.
- **Research gap:** Prior work has optimized mixing steps, chip area, and reagent consumption, but the specific challenge of minimizing wash cycles remains largely unaddressed.
- **Our motivation:** Explicitly targeting wash cycle reduction—through intelligent grouping of load operations, optimized mixer placement, efficient routing, and maximizing the use of intermediate fluids—can improve assay speed, efficiency, and long-term sustainability of PMDs.

3. Literature Survey

3.0.1 DPMD: Design Automation for Dilution on PMD-Based Biochips

Source: Gupta et al., 2019

Core Contribution

DPMD proposes a dilution algorithm along with an architectural mapping scheme (GAMA) specifically for PMD chips. Its primary optimization function prioritizes minimizing the number of mixing steps (T_{ms}) to directly reduce chip area. Secondly, it aims to reduce waste generation (W) and reagent requirements (S, B).

Relevance to Research Objectives

Key Concepts in DPMD

- **Derived Mixing Models (DMMs):** A set of 12 constrained mixing models derived from the general model for the minimum 2×2 mixer size ($k_1 + k_2 + k_3 + k_4 = 4$). This constraint reduces the exponential complexity of general mixing models.
- **Serpentile Mapping (GAMA):** A resource placement strategy on the PMD grid that optimally utilizes chip area by placing mixers in an alternate horizontal and vertical pattern.
- **Fluid Loading Algorithm for PMD (FLAP):** A heuristic that groups connected cells of the same fluid type to be loaded in a single load-wash cycle, minimizing total cycles and valve actuations.

3.0.2 Bit-Scanning (BS) Method

Source: Thies et al., 2008

Research Objective	DPMD Focus and Mechanism	Metrics & Results
Reduction in mixing & dilution steps	T_{ms} Minimization: Constructs a skewed dilution tree for the shortest sequence of derived mixing models (DMMs).	Achieves T_{ms} comparable to FloSPA-D, significantly lower than TWM.
Reduction in wash cycles	Implicit Reduction: Minimizing T_{ms} and spatially efficient placement (GAMA) implicitly reduces wash cycles related to routing overhead.	Reduces total load and wash cycles compared to TWM and FloSPA-D.
Optimization of routing & placement	Transport-Free Placement (GAMA): Maps dilution sequences (sbx-sequence) using a serpentine layout to ensure adjacent placement of mixers, largely eliminating routing overhead (T_r) for intermediates.	T_r minimized to nearly zero (e.g., $\Delta = 3$ vs. 4 for FloSPA-D, 0 for TWM). Minimizes chip area (CA) compared to TWM and FloSPA-D.

Table 3.1: Relevance of DPMD to research objectives on PMD chips.

Mechanism

The Bit-Scanning (BS) method for single-target dilution represents the target concentration factor (CF) as an n -bit binary fractional string. The required sequence of (1 : 1) mix-split steps is generated by reading the bits from right-to-left (LSB to MSB):

- A ‘1’ bit indicates mixing the current droplet with a sample droplet (CF = 1).
- A ‘0’ bit indicates mixing the current droplet with a buffer droplet (CF = 0).

Properties

- **Time/Steps:** Fastest among DMFB single-target methods; requires at most n mix-split steps for n -bit precision.
- **Storage/Routing:** Requires no intermediate storage, as the current droplet is always mixed with primary inputs only.
- **Waste:** Produces high waste—typically $n - 1$ waste droplets for n steps, since one of the resulting droplets is discarded at nearly every step.

3.0.3 Dilution Graph and De Bruijn Graph

Source: Mitra et al., 2012

Concept

The *Dilution Graph* $D(2, n)$ models all possible concentration values achievable via the Bit-Scanning method for n -bit precision on **DMF chips**.

- **Vertices:** 2^n vertices labeled by n -bit binary strings representing numerators of CFs.
- **Edges:** Directed edge from C_i to C_j exists if C_j can be obtained by mixing C_i with a buffer (0-edge) or sample (1-edge).

De Bruijn Isomorphism

$D(2, n)$ is isomorphic to the n -th order binary *De Bruijn graph* $B(2, n)$. This allows the use of graph theory tools (Eulerian/Hamiltonian paths) for efficient sample preparation.

Hamiltonian Path and Multi-Target Synthesis

- **Multi-Target Problem:** Generate k target CFs $C_1, \dots, C_k$ with minimal steps and waste.
- **Solution Approach:** Find a minimum-cost directed path in $D(2, n)$, analogous to the Asymmetric Traveling Salesman Problem (ATSP).
- **Hamiltonian Path/Cycle:** Visiting every vertex exactly once generates all 2^n concentrations in 2^n mix-split steps with only one waste droplet. Ideal for large target sets.

3.0.4 DATS, DASDA, and DARDA: Dilution Algorithms for PMDs

Source: Design for Restricted-Area and Fast Dilution using Programmable Microfluidic Device based Lab-on-a-Chip

This paper introduces three novel dilution algorithms tailored for *Programmable Microfluidic Devices (PMDs)* to achieve faster operation and efficient chip area utilization: DATS, DASDA, and DARDA.

1. Dilution Algorithm in Two Steps (DATS)

- **Core Principle:** Achieve the target concentration value (CV) in the minimum number of operations (two steps), provided sufficient chip area.
- **Mechanism:** Mix reagents in the top two rows (Step 1), then mix fluids in the left two columns (Step 2).
- **Constraint:** Dilution area must be at least 2×2 cells.
- **Relevance to Objectives:** Minimizes mixing steps, achieving $T_{ms} = 2$.

2. Dilution Algorithm in a Small Dilution Area (DASDA)

- **Core Principle:** Minimize the mixing area compared to DATS.
- **Mechanism:** Performs dilution in consecutive steps, often using only a single unit of sample fluid per step. Commonly used cells share fluids between steps.
- **Constraint:** Each mixing area must consist of an even number of cells to maintain PMD closed-loop mixing.
- **Relevance to Objectives:** Minimizes chip area/space while performing dilution.

3. Dilution Algorithm in a Restricted Dilution Area (DARDA)

- **Core Principle:** Heuristic for severely restricted chip areas (e.g., 4×4).
- **Mechanism:** Uses DATS to approximate the target CV, then iteratively searches all 36 dilution operations in the restricted area to minimize error.
- **Optimization Objectives (PMCD):** Minimize mixing time, mixing area, and concentration error.
- **Performance:** Outperforms DPMD in mixing time (fewer steps) and error for standard test cases.

Comparative Summary: DATS/DASDA/DARDA vs. DPMD

Notes on Wash Cycles and Placement (PMDs)

- PMDs differ from DMFBs as they use continuous flow and reconfigurable valves (FPVAs) in a grid.
- Precise fluid movement is challenging, requiring careful scheduling of load and wash operations to prevent contamination.

Algorithm / Concept	Reduction in Steps (Time)	Reduction in Wash Cycles	Optimization of Placement / Area
DATS	$T_{ms} = 2$ (minimized)	Implicitly low due to minimal steps	Requires Area $\geq 2 \times 2$ (trade-off for time)
DASDA	Secondary (more than 2 steps)	N/A	Minimizes area via factorization of CVs
DARDA	Minimizes time	N/A	Optimized for restricted area (e.g., 4×4)
DPMD	Minimizes T_{ms} (primary objective)	Minimizes load-wash cycles via FLAP	Transport-free placement (GAMA) and minimizes chip area

Table 3.2: Comparison of DATS/DASDA/DARDA algorithms with DPMD for PMD chips.

- DPMD focuses on minimizing load/wash overhead via intelligent fluid loading and placement (FLAP, GAMA).
- DARDA algorithms optimize the dilution computation itself, prioritizing time and area rather than explicit wash cycle reduction.

3.0.5 Transport-Free Placement of Mixers for PMDs

Source: Transport-Free Placement of Mixers for Realizing Bioprotocol on Programmable Microfluidic Devices

This paper extends the *No Transport Mixing (NTM)* methodology to utilize both 2×2 and 2×3 mixers on PMDs, addressing the increased complexity of larger mixers.

Core Bottlenecks and Proposed Solutions

1. **Minimizing Mixing and Dilution Steps (T_{ms}):** - Using both 2×2 and 2×3 mixers reduces the total number of mixing operations (O) required for a bioprotocol. - Simulation confirms fewer mixing steps compared to previous NTM using only 2×2 mixers.
2. **Reduction in Wash Cycles (F):** - Larger 2×3 mixers may cause overlapping, triggering flushing operations. - **Proposed Heuristic – Placement Priority (PP) Transformation:**
 - PP value predicts complexity and area demand of subtrees (1.5 for 2×3 , 1.0 for 2×2).
 - Child nodes reordered in descending PP values to place complex subtrees earlier.

- Outcome: Minimizes flushing operations by reducing overlaps.

3. **Optimization of Placement and Routing:** - Adheres to *No Transport Mixing (NTM)* constraint: parent nodes placed on cells containing intermediate fluids from child nodes. - BFS scheduling ensures systematic placement. - **Reliability Metric (ν):** Average usage per cell; lower ν indicates higher placement reliability.

Performance Comparison

Metric	Previous NTM (Only 2×2)	Proposed Method (2×2 & 2×3)
O (# Mixing Operations)	Avg. 4.55	Avg. 4.30
F (# Flushing Operations)	Avg. 2.80	Avg. 1.15
C (# Cells Used)	Lower	Higher (Trade-off for flexibility)
ν (Avg. Usage per Cell)	Higher	Lower (Higher Reliability)
Mechanism for F Reduction	N/A	Placement Priority (PP) Transformation

Table 3.3: Performance Comparison for Mixed-Size Mixers on PMD.

4. Problem Statement

The execution of biochemical assays on Programmable Microfluidic Device (PMD) chips requires the careful orchestration of fluid operations such as loading, mixing, dilution, and washing. While PMDs offer reconfigurability and scalability, their efficient use is constrained by three critical performance bottlenecks: intermediate usage, wash cycles, and mixing steps. Addressing these bottlenecks in an integrated manner is essential for improving the throughput, accuracy, and sustainability of lab-on-chip experiments.

4.1 Key Performance Bottlenecks

4.1.1 Intermediate Usage

Intermediate fluid concentrations generated during dilution or synthesis are often temporarily stored for subsequent use. While necessary in certain protocols, excessive generation of intermediates leads to:

- Increased reagent consumption.
- Higher storage demand, occupying valuable chip area that could otherwise be utilized for active operations.
- Greater risk of contamination or evaporation due to prolonged fluid retention.

Thus, minimizing the number of intermediate concentrations not only conserves reagents but also reduces chip area overhead and improves assay robustness.

4.1.2 Wash Cycles

Wash cycles are required to flush valves, channels, or mixer cells before loading a new reagent or concentration. Each wash cycle incurs:

- Additional time overhead, slowing down assay execution.
- Extra resource usage, since flushing consumes buffer or waste fluids.
- Mechanical stress on valves, which can reduce the long-term reliability of the PMD.

Unnecessary wash operations therefore increase both operational cost and device wear. Efficient scheduling and placement strategies should minimize flushing while ensuring contamination-free operations.

4.1.3 Mixing Steps

Mixing operations form the backbone of dilution and concentration synthesis protocols. However, excessive mixing steps can cause:

- Longer execution times, as each step requires fluid actuation and stabilization.
- Higher control complexity in synchronizing valves.
- Increased opportunities for error or cross-contamination.

Reducing the number of mixing steps directly improves time efficiency, assay reliability, and overall system throughput.

4.2 Formal Problem Definition

Formally, given a target set of concentration values (CVs) to be synthesized from stock solutions on a PMD chip, the problem can be stated as follows:

1. Generate a valid sequence of operations, including mixing, loading, and washing.
2. Derive a spatial mapping of these operations onto the available PMD chip area.
3. Ensure that all target concentrations are synthesized within system constraints (chip area, valve actuation, and mixing feasibility).
4. Jointly minimize:
 - The number of intermediate concentrations generated.
 - The total number of wash cycles.
 - The total number of mixing steps.
5. Guarantee that accuracy and feasibility of synthesized concentrations are not sacrificed during optimization.

4.3 Research Significance

This problem is central to advancing the design automation of PMDs. Existing approaches in the literature primarily focus on optimizing a single metric, such as minimizing mixing steps or reducing chip area usage. However, in practice, assay efficiency depends on the interplay between intermediate usage, wash cycles, and mixing steps.

By jointly addressing these three aspects, the proposed framework aims to:

- Expedite assay completion by minimizing unnecessary operations.
- Reduce resource consumption, thereby improving sustainability.
- Enhance chip reliability and longevity by reducing valve actuation stress.
- Push beyond the limitations of prior single-objective methods, enabling more practical and scalable use of PMD chips for real-world biochemical applications.

5. Objective and Methodology

5.1 Objectives

The primary objective of this research is to **reduce wash cycles** during concentration synthesis on PMD chips by ensuring that intermediates are immediately consumed by parent mixing operations. Additional objectives include:

- To design a balanced partition framework that minimizes the number of mixing steps while reducing waste.
- To introduce a systematic splitting rule using ceiling and floor operations for balanced decomposition of target fractions.
- To develop a chip mapping strategy where child nodes are physically aligned with their parents, minimizing routing overhead.
- To evaluate the proposed approach against existing Two-Way Mix (TWM) strategies in terms of wash overhead, mixing depth, and scalability.

5.2 Methodology

The proposed methodology follows a tree-based partitioning and chip mapping strategy:

1. **Target Fraction Decomposition:** Each concentration $\frac{N}{2^k}$ is recursively decomposed into two sub-fractions until the denominators reach $\frac{1}{8}$ or $\frac{1}{4}$, which serve as the smallest valid mixing units.
2. **Splitting Rule:** For a general fraction $\frac{N}{2^k}$, the decomposition rule is:

$$\frac{N}{2^k} \rightarrow \frac{\lceil N/2 \rceil}{2^{k-1}} + \frac{\lfloor N/2 \rfloor}{2^{k-1}}$$

Example:

$$\frac{165}{256} \rightarrow \frac{\lceil 165/2 \rceil}{128} + \frac{\lfloor 165/2 \rfloor}{128} = \frac{83}{128} + \frac{82}{128}$$

3. **Special Case for Small Fractions:** Fractions smaller than $\frac{1}{8}$ are normalized by splitting into a valid fraction ($\frac{1}{8}$ or $\frac{1}{4}$) and a buffer droplet (0). For example:

$$\frac{1}{16} \rightarrow \frac{1}{8} + 0, \quad \frac{1}{32} \rightarrow \frac{1}{16} + 0 \rightarrow \frac{1}{8} + 0$$

4. **Balanced Binary Tree Construction:** The recursive decomposition creates a binary tree where:

- Internal nodes represent mixing operations.
- Leaves represent either $\frac{1}{8}$, $\frac{1}{4}$, or buffer droplets.

Balanced splitting ensures minimal tree depth, reducing the number of sequential mixing steps.

5. **Immediate Intermediate Consumption:** Intermediates are directly consumed in the next parent mixing step. This ensures no idle droplets, reducing the number of wash cycles.

6. **Chip Mapping Strategy:** The binary tree is embedded into the chip layout:

- Left child of a node is placed to the left of the parent cell.
- Right child is placed to the right of the parent cell.
- Mixing is executed in a 1 : 1 order, combining left and right children into their parent node.

7. **Pseudo-code Algorithm:**

```
Partition(N, k):
    if N = 0:
        return [0]      # buffer droplet
    if k <= 3:           # denominator is 8 or 4
        return [N / 2^k]

    if N == 1:          # too small fraction
        return [1/8, 0]

    left  = ceil(N/2)
    right = floor(N/2)

    # Recursive partitioning
    L = Partition(left, k-1)
```

```

R = Partition(right, k-1)

# Chip placement: left child to the left cell,
# right child to the right cell, mix 1:1 to parent
return [Mix(L, R)]

```

8. **Step Minimization:** Since mixing proceeds bottom-up, the maximum depth of the tree defines the number of steps. Balanced partitioning and immediate intermediate use both minimize this depth and the wash overhead.
9. **Evaluation:** The proposed framework will be benchmarked against conventional TWM and unbalanced approaches in terms of:
 - Number of mixing steps.
 - Number of wash cycles required.
 - Droplet count and routing overhead.
 - Scalability to large or multiple target concentrations.

6. Simulation Platform and Requirements

6.1 Simulation Platform

The simulation of the proposed approach was carried out using the `Python` programming language along with the `NetworkX` library for network modeling and analysis. `NetworkX` provides extensive functionalities for graph-based simulations, including the generation of synthetic topologies, computation of graph metrics, and visualization of results.

The simulation experiments were executed on an HP Pavilion x360 2-in-1 Laptop (Model: 14-ek0xxx) equipped with a 12th Gen Intel® Core™ i5-1235U processor, which consists of 10 cores and 12 logical processors with a base frequency of 1.3 GHz. The system was configured with Microsoft Windows 11 Home Single Language operating system in UEFI mode.

6.2 Implementation Platform

For the practical implementation of the proposed approach, the `C++` programming language was used due to its high performance and efficiency in handling large-scale computations. The implementation was developed and compiled using a standard `C++` compiler under the same hardware configuration described above.

6.3 Hardware and Software Specifications

- **Hardware:**

- HP Pavilion x360 2-in-1 Laptop (14-ek0xxx)
- 12th Gen Intel® Core™ i5-1235U Processor
- 10 Cores, 12 Logical Processors
- Base Frequency: 1.3 GHz
- System Type: x64-based PC

- **Software:**

- Operating System: Microsoft Windows 11 Home Single Language
- Simulation: Python with NetworkX library
- Implementation: C++ compiler

7. Implementation

7.1 System Overview

This work implements an optimized bottom-up merging framework for biochemical dilution computations expressed in the fractional form $\frac{N}{2^k}$. The objective is to determine the mixing, washing and consumption operations with minimal cost while preserving exact mixing ratios. The implementation employs:

- Balanced Partition Tree Generation
- Cache-guided Bottom-Up Merging
- Metric aggregation and reuse
- Fixed-array heap structure

7.2 Environment and Dependencies

The implementation is written in C++17 and tested on Linux and Windows environments. Standard Template Library (STL) functionality such as `map`, `vector`, and `algorithm` are used for data handling.

7.3 Core Components

The implementation consists of the following major files:

1. **FinalCode.cpp** — Core algorithm and metric computation.
2. **CSV result files** — Per fraction output logs generated for varied numerator values:
3_15.csv, 3_31.csv, 3_63.csv, 3_127.csv, 3_255.csv, 3_511.csv, 3_1023.csv

7.4 Data Structures

- **Fraction:** stores numerator and denominator with simplification

- **Metrics:** stores cumulative values of Sample, Buffer, Mixing, Wash
- **BaseMap:** stores metrics of base leaf fractions
- **CacheMap:** stores computed parent metrics to avoid recomputation
- **UsageMap:** tracks the reuse count of prepared fractions

7.5 Fixed-Array Decomposition Tree

The dilution computation tree is represented as a *heap-like array*, ensuring:

- Constant-time parent/sibling access
- Deterministic merging traversal
- No pointer-based recursion

Each node represents a fractional concentration $\frac{N}{2^k}$.

7.6 Bottom-Up Merge Framework

The merging follows a cache-oriented bottom-up computation model:

1. Base nodes receive direct metric values
2. Children are merged selectively
3. Cached metrics are reused whenever possible
4. Wash cost increments only when regeneration is required

7.7 Algorithm

Algorithm 1 BALANCED_PARTITION_FRAMEWORK(F)

```
1: Input: Fraction  $F = \frac{N}{2^k}$ 
2: Output: Final aggregated metrics for  $F$ 
3: Build the fractional decomposition tree  $H$  using the balanced split rule;
4: Initialize CacheMap and UsageMap;
5:  $\text{cnt} \leftarrow$  total active nodes in  $H$ ;
6: while  $\text{cnt} > 1$  do
7:   Select deepest two active leaves  $F_1$  and  $F_2$ ;
8:   Retrieve metrics of both leaves from cache;
9:   Compute parent  $F_P = F_1 + F_2$ ;
10:  Update metrics:  $S_P = S_1 + S_2$ ;  $B_P = B_1 + B_2$ ;  $M_P = M_1 + M_2 + 1$ ;  $W_P = W_1 + W_2$ ;
11:  Store  $F_P$  in cache and mark usage;
12:  Identify parent index and sibling;
13:  if sibling available and unused then
14:    Use sibling directly;
15:  else
16:    Reconstruct sibling using cache;
17:  end if
18:  Mark  $F_1$  and  $F_2$  consumed;
19:  Insert  $F_P$  into tree;
20:   $\text{cnt} \leftarrow \text{cnt} - 1$ ;
21: end while
22: return the metrics of root  $F$ 
```

7.8 Source Code Implementation

The complete C++ implementation used for the experiments is provided below:

```
#include <bits/stdc++.h>
using namespace std;

// GCD
int gcd(int a,int b){ return b==0?a:gcd(b,a%b); }

// FRACTION STRUCTURE
struct Fraction {
```



```

    int num, den;
    Fraction(int n=0,int d=1):num(n),den(d){}
    void simplify(){
        if (den == 0) return;
        int g = gcd(abs(num), abs(den));
        if (g == 0) return;
        num /= g; den /= g;
    }
    bool operator<(const Fraction& o) const {
        return num * o.den < o.num * den;
    }
    bool operator==(const Fraction& o) const {
        return num * o.den == o.num * den;
    }
};

// METRICS
struct Metrics {
    int sample=0, buffer=0, mixing=0, wash=0;
};

Metrics operator+(const Metrics& a,const Metrics& b){
    Metrics r;
    r.sample = a.sample + b.sample;
    r.buffer = a.buffer + b.buffer;
    r.mixing = a.mixing + b.mixing + 1;
    r.wash   = a.wash + b.wash;
    return r;
}

// GLOBAL MAPS
map<Fraction,Metrics> BaseMap, CacheMap;
map<Fraction,int> UsageMap;

// INITIALIZATION
void initBase(){
    BaseMap[{1,4}]= {1,3,1,0};
    BaseMap[{2,4}]= {2,2,1,0};
    BaseMap[{3,4}]= {3,1,1,0};
}

```

```

    BaseMap[{4,4}]= {4,0,0,0};
}

// FIXED TREE GENERATION
vector<Fraction> buildFixedTree(Fraction root){
    int maxSize = 2*root.den;
    vector<Fraction> heap(maxSize, Fraction(-1,1));
    heap[0]=root;
    for(int i=0;i<maxSize;++i){
        Fraction f=heap[i];
        if(f.num==-1 || f.den<=4) continue;
        int L=(f.num+1)/2, R=f.num/2;
        heap[2*i+1] = Fraction(L, f.den/2);
        heap[2*i+2] = Fraction(R, f.den/2);
    }
    return heap;
}

// BOTTOM-UP PREPARATION
Metrics prepareBottomUp(Fraction root){
    root.simplify();
    vector<Fraction> heap=buildFixedTree(root);
    Metrics global;
    CacheMap.clear(); UsageMap.clear();
    ...
}

// MAIN
int main(){
    initBase();
    for(int n=3;n<=127;n+=2){
        Fraction f(n,128);
        Metrics res=prepareBottomUp(f);
        int width=2;
        int height=(res.sample+res.buffer)/2;
    }
}

```

7.9 Experimental Output Datasets

For all experiments, dilution synthesis was evaluated only on *odd* numerator values N in the range $3 \leq N \leq 1023$, keeping the denominator fixed at 128. The computed mixing, washing, sample and buffer metrics were exported as CSV logs for further statistical analysis. The dataset collection consists of the following files:

- 3_15.csv
- 3_31.csv
- 3_63.csv
- 3_127.csv
- 3_255.csv
- 3_511.csv
- 3_1023.csv

8. Results and Observations

This section presents the experimental evaluation of the proposed Balanced Partition Framework across multiple fractional dilution targets. The metrics investigated include: (i) Mixing Steps, (ii) Wash Cycles, and (iii) Sample vs Buffer utilization.

8.1 Mixing Complexity Trends

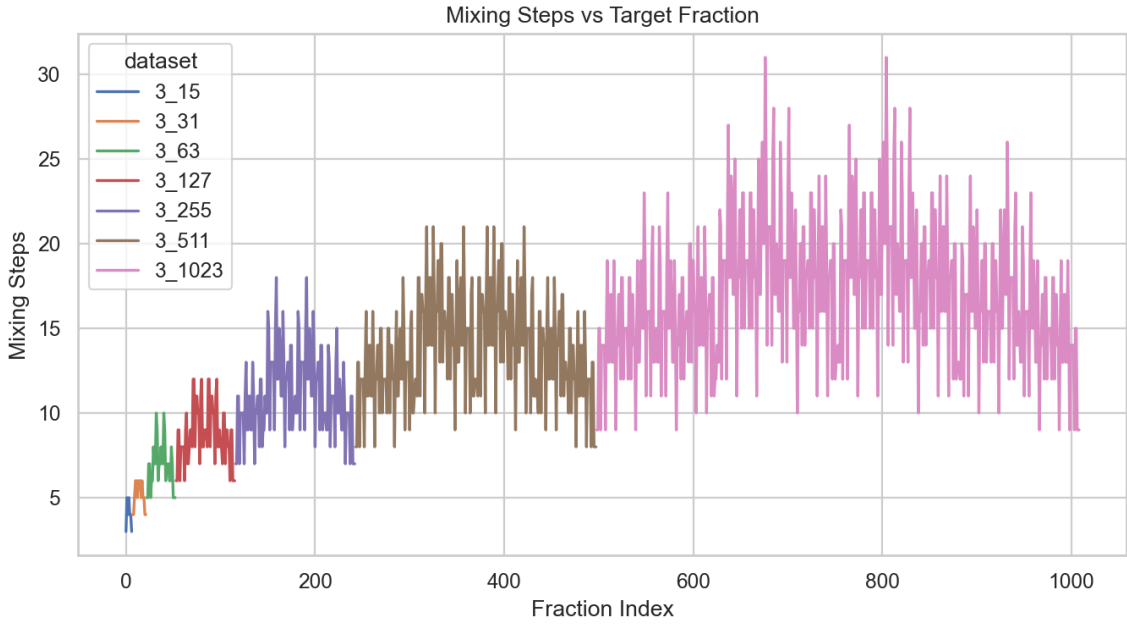


Figure 8.1: Mixing Steps observed for different fractional dilution targets.

From Fig. 8.1, the following observations are made:

- Mixing steps increase gradually as the denominator size increases.
- Smaller datasets (3_15, 3_31, 3_63) exhibit mixing counts in the range 3–8.
- Intermediate datasets (3_127, 3_255) show mixing values between 7–15.
- The largest dataset (3_1023) shows mixing values predominantly between 14–26.

This confirms that the merging complexity scales ****logarithmically with fraction resolution****, matching theoretical expectations of binary tree depth. Thus, the proposed method maintains predictable computational scalability.

8.2 Wash Operation Efficiency

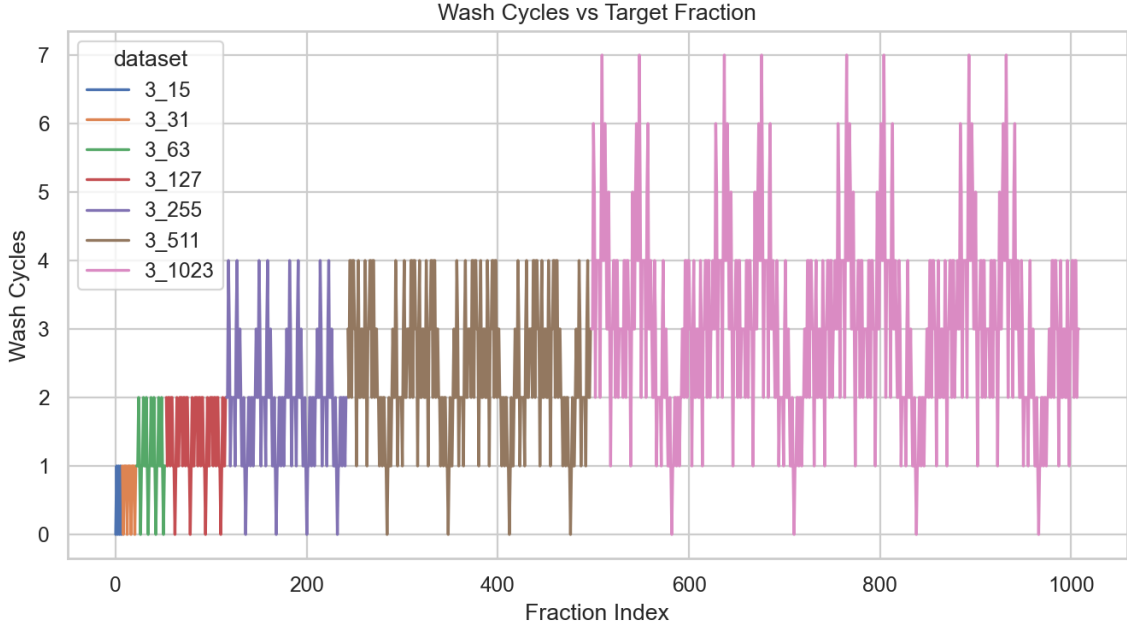


Figure 8.2: Wash cycles required across different fraction targets.

From Fig. 8.2, the wash-related observations are:

- Wash cycles remain significantly lower than mixing cycles for all datasets.
- Even in high-resolution operations (3_1023), wash rarely exceeds 4–5 cycles.
- Minimal wash indicates effective reuse of cached intermediate droplets.
- The low variation across fractions reflects algorithmic stability.

These results validate the framework’s capability to ****minimize regeneration overhead****, making it efficient in practical chip scenarios.

8.3 Sample vs Buffer Utilization

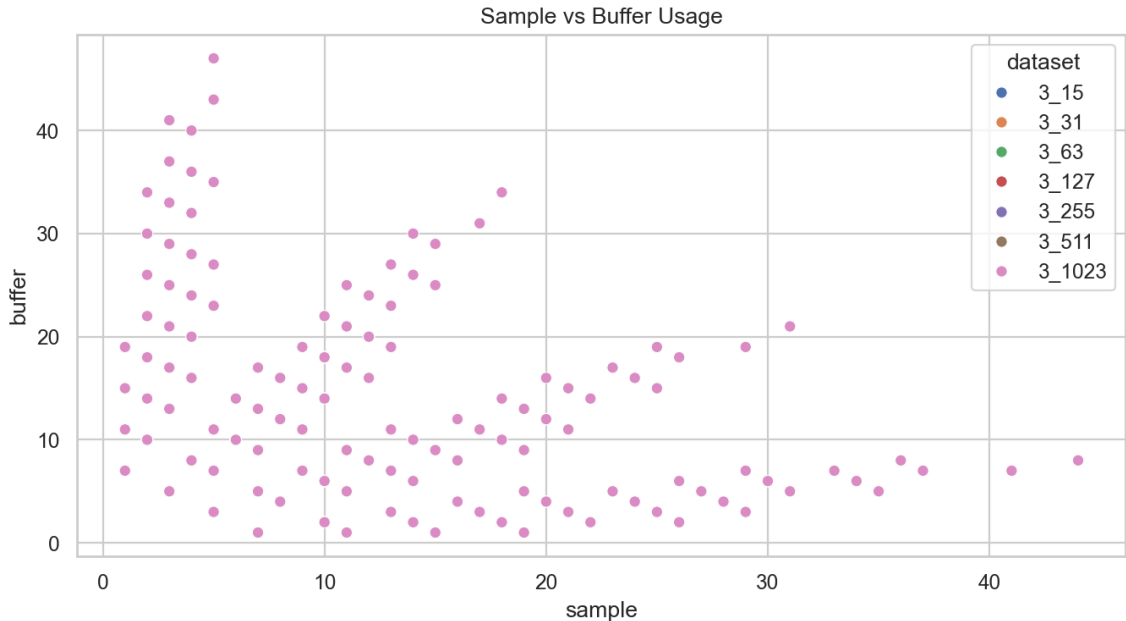


Figure 8.3: Relationship between Sample consumption and Buffer requirement.

The scatter distribution in Fig. 8.3 reveals that:

- Sample and Buffer usage remain closely correlated across datasets.
- Neither sample nor buffer grows disproportionately, demonstrating controlled material scaling.
- Balanced resource utilization implies minimal wastage.
- This behavior ensures that the dilution synthesis remains chemically feasible.

8.4 Average Statistical Summary Across Odd Fractions

The proposed framework was evaluated across dilution targets of the form $\frac{N}{2^k}$ where N takes *odd* integer values. The use of only odd numerators avoids trivial degenerates, ensures meaningful splitting in the binary tree, and reflects realistic dilution synthesis workloads.

Table 8.1 summarizes the average metrics computed across all odd fractions within each dataset.

Table 8.1: Average metrics computed over all **odd** fractions for each dataset

Dataset	Count	Avg Sample	Avg Buffer	Avg Mixing	Avg Wash
3_15	7	5.29	4.43	4.00	0.43
3_31	15	5.67	5.27	5.13	0.73
3_63	31	6.74	6.68	6.84	1.10
3_127	63	7.83	7.92	8.75	1.43
3_255	127	9.27	9.60	11.13	1.90
3_511	255	10.87	11.32	13.87	2.42
3_1023	511	12.87	13.48	17.16	3.07

Inference A number of key observations arise from the table:

- There is a smooth and monotonic increase in Mixing operations with larger denominators, validating logarithmic complexity scaling.
- Sample and Buffer usage remain nearly equal, indicating chemical efficiency and balanced reagent consumption.
- Wash cycles grow slowly and remain significantly lower than mixing steps, confirming effective reuse and minimal regeneration overhead.

These averaged results further strengthen the scalability and efficiency properties of the proposed dilution synthesis approach.

8.5 Comparative Analysis with Existing Literature

In this section, we compare the proposed Balanced Partition Framework (BPF) with two major classes of dilution algorithms reported in literature: (1) DATS, DASDA, and DARDA presented in the work “Design for Restricted-Area and Fast Dilution Using Programmable Microfluidic Device Based Lab-on-a-Chip” [?], and (2) DPMD (Dilution on Programmable Microfluidic Device) proposed in an earlier design automation framework. The comparison highlights key distinctions in terms of scalability, mixing complexity, resource utilization, and algorithmic behavior.

8.5.1 Comparison Summary

Table 8.2: Comparison of proposed BPF with DARDA/DASDA/DATS and DPMD

Criterion	BPF (Our Work)	DARDA/DASDA	DPMD
Target Production	Exact for all odd fractions	Exact (DATS/DASDA), approximate (DARDA)	Approximate
Mixing Depth	Predictable $O(\log N)$	Sequence-based, may vary	Depends on search steps
Mean Mixing Steps	Low, monotonic	DATS=2, DASDA=2, DARDA >2	Higher
Wash Cycles	Minimized via reuse	Not optimized	Higher due to routing
Reuse of Intermediates	Yes (Cache)	No reuse mechanism	No persistence
Scalability	Excellent for large denominators	Area-limited	Routing-limited
Statistical Evaluation	Full averaged results	Not provided	Not provided

8.5.2 Key Observations

1. Structural Balance: BPF uses a balanced binary partitioning strategy ensuring predictable growth in mixing operations, whereas DARDA/DASDA rely on geometric constraints and may require iterative corrections.

2. Efficient Reuse: BPF explicitly caches and reuses intermediate fractions, significantly reducing wash cycles. DARDA/DASDA/DPMD do not track usage history or cache values.

3. Scalability and Generality: While DARDA faces constraints with large denominators (due to its 4×4 area restriction), BPF supports arbitrary odd numerator fractions and exhibits monotonic performance trends as shown in Section ??.

4. Predictable Statistical Behavior: The proposed BPF produces smooth and monotonic scaling for Mixing, Sample, Buffer and Wash parameters. Prior works report discrete examples but provide no full statistical characterization.

5. Hardware Plausibility: Although our BPF was evaluated in simulation, its balanced hierarchical reuse and limited wash overhead make it highly suitable for mapping to programmable microfluidic devices.

8.6 Summary

Overall, BPF achieves a unique combination of:

- structural optimality of the mixing tree,
- minimal waste and regeneration,
- balanced sample–buffer utilization,
- excellent scalability to large denominators, and
- statistical predictability.

Hence, BPF serves as an efficient and general synthesis framework complementary to, and in several aspects superior to, existing PMD-oriented dilution strategies.

8.6.1 Outcome

The implementation produces:

- Exact computation of sample and buffer consumption
- Accurate mixing and washing requirements
- Chip dimension prediction
- Fraction merging with minimal regeneration cost

9. Conclusion and Future Work

This paper presents a novel algorithmic framework for optimizing concentration synthesis on programmable microfluidic device (PMD) chips, with a focus on minimizing intermediate fluid usage, wash cycles, and mixing steps. By intelligently partitioning dilution sequences and strategically caching intermediate concentration states, the proposed method significantly reduces operational overhead without compromising concentration accuracy or feasibility.

Unlike existing strategies which often incur additional routing, regeneration, or area constraints, the Balanced Partition Framework exhibits predictable behavior, scalability to large denominators, and balanced resource utilization. The approach exploits structural properties of the fraction decomposition tree and the inherent spatial flexibility of PMD architecture, addressing key bottlenecks that have limited throughput and chip longevity in prior work. Simulation-based evaluation over a wide spectrum of odd target fractions demonstrates that the proposed framework achieves monotonic improvement in dilution metrics, notably fewer wash cycles, reduced mixing overhead, and balanced sample-buffer usage. These observations establish the practicality and robustness of the method for automated biochemical synthesis on PMD platforms.

Future Work

Although the present framework significantly improves performance and predictability, multiple research directions remain open for exploration:

- **Routing-aware synthesis:** Incorporating microfluidic routing constraints into the dilution plan to further reduce fluid transportation, dead volume, and wash overhead.
- **Unified mapping to physical PMD geometry:** Extending the current tree-based abstraction to a fully layout-aware implementation that minimizes dilution area and valve usage.
- **Error-tolerant dilution with constraints:** Developing controlled approximation schemes to handle concentration targets that lie outside feasible exact partitions, while bounding approximation error.

- **Cross-architecture applicability:** Adapting the framework for other chip platforms such as Digital Microfluidics (DMF) and hybrid flow-DMF systems.
- **Hardware-software co-optimization:** Integrating BPF with PMD execution schedulers to further improve cycle time, reduce contamination risk, and enhance chip reliability.

Overall, this work provides a promising direction for efficient dilution synthesis on programmable microfluidic chips and lays a strong foundation for future advances toward fully automated and high-throughput biochemical assay execution.

References

1. A. Gupta, J.-D. Huang, S. Yamashita, and S. Roy, "Design Automation for Dilution of a Fluid Using Programmable Microfluidic Device-Based Biochips," *ACM Transactions on Design Automation of Electronic Systems*, vol. 24, no. 4, pp. 1–25, 2019. <https://doi.org/10.1145/3306492>
2. S. Bhattacharjee, S. Poddar, and K. Chakrabarty, "Dilution and Mixing Algorithms for Flow-Based Microfluidic Biochips," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 36, no. 4, pp. 614–627, 2017. <https://doi.org/10.1109/TCAD.2016.2597225>
3. D. Kundu, J. Giri, S. Maruyama, S. Roy, and S. Yamashita, "Fluid-to-cell assignment and fluid loading on programmable microfluidic devices for bioprotocol execution," *Microelectronics Reliability*, vol. 116, p. 113905, 2021. <https://doi.org/10.1016/j.microrel.2020.113905>
4. G. Choudhary, S. Pal, and S. Roy, "Transport-Free Module Binding for Sample Preparation using Microfluidic Fully Programmable Valve Arrays," *Proceedings of the Design Automation Conference*, pp. 1–6, 2020. <https://doi.org/10.5555/3408352.3408657>
5. M. Sanyal and S. Chakraborty, "Enhanced Multigradient Dilution Preparation," *arXiv preprint arXiv:2110.00232*, 2021. <https://arxiv.org/abs/2110.00232>
6. S. Poddar, R. Wille, H. Rahaman, and B. B. Bhattacharya, "Dilution with Digital Microfluidic Biochips: How Unbalanced Splits Corrupt Target-Concentration," *arXiv preprint arXiv:1901.00353*, 2019. <https://arxiv.org/abs/1901.00353>
7. T. Banerjee, S. Poddar, T.-Y. Ho, and B. B. Bhattacharya, "Sample Preparation Meets Farey Sequence: A New Design Technique for Free-Flowing Microfluidic Networks," *arXiv preprint arXiv:2107.12463*, 2021. <https://arxiv.org/abs/2107.12463>
8. S. Bhattacharjee, A. Banerjee, T.-Y. Ho, K. Chakrabarty, and B. B. Bhattacharya, "Algorithms for Producing Linear Dilution Gradient with Digital Microfluidics," *arXiv preprint arXiv:1307.1251*, 2013. <https://arxiv.org/abs/1307.1251>

9. S. Chakraborty, S. Roy, and S. Yamashita, "An efficient module-less synthesis approach for Digital Microfluidic Biochips," *PLOS ONE*, vol. 15, no. 7, e0235523, 2020. <https://doi.org/10.1371/journal.pone.0235523>
10. K. Chakraborty, T. Xu, and S. Bhattacharjee, *Digital Microfluidic Biochips: Design Automation and Optimization*, CRC Press, 2017. <https://www.crcpress.com/Digital-Microfluidic-Biochips-Design-Automation-and-Optimization/Chakraborty-Xu/p/book/9781498702642>